Tool Calling, Guardrails and Generative UI

> Part of the Conversational Agent curriculum. This document covers using an MCP server as a tool source, classifying requests before acting on them, bounding the tool-calling loop, two different kinds of caching, streaming progress, mapping results to real UI components instead of freeform generated markup, and the "structural" guardrails that keep required disclosures, numbers and links out of the model's hands.

1. Using an MCP server as an agent's tool source

Rather than hand-writing each tool an agent can call, connect to an MCP server as an MCP client and let it hand the agent its live tool list. This is exactly the M×N integration problem MCP was built to solve, from the client side: an agent built this way gets a maintained, independently versioned tool surface without duplicating its schemas by hand, and — just as importantly — inherits whatever guardrail logic already lives at the MCP layer for free. An agent doesn't need to re-implement, say, fuzzy name matching on a country argument, because a well-designed MCP tool already returns a clear, structured error the model can react to and retry on its own; see MCP Server, Tool Design, §3.

A client-adapter library typically handles converting MCP tool schemas into whatever shape the calling LLM API expects, and manages the underlying session — worth using rather than hand-rolling that plumbing inside agent code.

2. Guardrails: classify before you act

Not every user turn should reach the tool-calling loop at all. A first **routing** step — often a small, cheap structured-output LLM call, sometimes a fixed rule needing no LLM call at all — sorts each incoming request into a handful of fixed categories before anything else happens:

Classification	Typical handling
Off-topic	Decline with a fixed, pre-written response — no LLM call needed once detected at all. A canned refusal is cheaper, faster, and can't drift in wording the way a regenerated-every-time refusal can.
Subjective / opinion-shaped	Decline the subjective framing specifically, but offer a nearby question the underlying data actually *can* answer, rather than a flat no.
General factual, answerable from the model's own knowledge	Answer directly, with no tool call — constrained to stay factual rather than opinion-shaped, but genuinely doesn't need a data lookup.
A genuine data question	Proceed into the full tool-calling loop.

Running this classification **first**, as its own explicit node, keeps the rest of the graph simpler: the tool-calling loop only ever has to handle "a real data question," not also decide mid-loop whether the question was ever data-shaped to begin with.

3. Bounding the tool-calling loop

The model↔tools cycle (Core Concepts, §5) is the one part of the graph that can genuinely run away — nothing but the model's own judgment naturally stops it. Add an explicit hard cap on tool calls per turn, checked on the conditional edge leaving the tools node:

```
MAX_TOOL_CALLS_PER_TURN = 6

def route_after_agent(state):
    if not agent_requested_tool_call(state.messages[-1]):
        return "compose_response"
    if state.tool_call_count >= MAX_TOOL_CALLS_PER_TURN:
        state.scope_notes.append(
            f"Stopped after {MAX_TOOL_CALLS_PER_TURN} tool calls -- this response may be based on
        partial data."
        )
        return "compose_response"
    return "tools"
```

Treat hitting the cap as a **normal, handled outcome** — finish with whatever's been gathered and surface an explicit note that the answer may be based on partial data — rather than as an error state that aborts the turn.

A cache hit still has to count toward this cap (§4 explains why the cache exists at all): exempting cached calls would open a loophole where a stuck model spams "free" repeated calls without ever tripping the guard. The cache exists to save network round-trips and API cost — it doesn't grant extra loop iterations.

4. Two different kinds of caching, easy to conflate

Kind	Scope	Purpose
App-level tool-result cache	One conversation thread	Avoid re-issuing an identical tool call already answered earlier in the same conversation
Provider-side prompt caching	One LLM API's own infrastructure	Avoid re-billing/re-processing an identical *request prefix* (e.g. a large, unchanging tool-schema list plus a system prompt) across repeated calls

App-level cache, keyed by `(tool_name, normalized_args)` — normalize any list-valued argument (sort it) before hashing, so two semantically identical calls with reordered arguments (`["China", "India"]` vs. `["India", "China"]`) still hit the same key:

```
def cache_key(tool_name: str, args: dict) -> str:
    normalized = {k: sorted(v) if isinstance(v, list) else v for k, v in args.items()}
    return f"{tool_name}:{json.dumps(normalized, sort_keys=True)}"
```

Provider-side prompt caching is a separate mechanism some LLM APIs expose (a cache-control marker attached to part of the request), letting a repeated *prefix* — a large tool catalog's schemas plus a system prompt,

for instance — be served from cache on repeat calls within the same loop, or across separate user requests that share the same tool set. (Self-hosted inference servers often cache the prompt prefix automatically with no marker at all — check what your provider does before adding code for it.) It's worth enabling specifically on whichever request component is both **large** and **identical across calls** — a full tool schema list is a common, good candidate; small, cheap, single-shot prompts elsewhere in the graph usually sit below whatever minimum-size floor makes caching worthwhile, and marking them adds a write-side cost premium with no meaningful read-side payoff.

These two caches solve genuinely different problems and live at different layers — don't conflate "the agent already asked this" (app-level) with "the provider already saw this exact request prefix" (provider-side).

5. Streaming progress mid-loop

A tool-calling loop with several iterations can take long enough that a UI needs to show *something* before the final answer arrives. Stream per-step progress events as the loop runs — most graph-orchestration frameworks offer a streaming-invocation mode for exactly this — rather than only returning the fully composed result at the very end and leaving a caller blank in between.

Prefer **templated, per-tool-name progress labels** ("Fetching historical emissions for India…") over asking the model to generate a running description of what it's doing at each step: templated labels are deterministic, consistent in phrasing, and don't cost an extra LLM call per step just to narrate a step that's already fully determined by which tool is running.

6. Generative UI: map results to real components, not generated markup

Once a turn's tool results are in hand, a separate step decides **how** to render them. The safest, most maintainable design is a **fixed lookup from tool name to UI intent** — "this tool's result is always a line chart," "that one is always a table" — rather than letting the model freely generate markup or improvise a layout per call.

A fixed mapping keeps the rendering surface exactly as wide as the set of components you've actually built and tested: there's no risk of a model inventing a component, a prop shape, or a layout nothing on the frontend understands how to render.

```
TOOL_TO_INTENT = {
    "get_historical_series": "chart",
    "get_comparison_table": "grid",
    "get_entity_profile": "card",
    "get_methodology_notes": "text",
}
```

Reserve an actual LLM judgment call for the rare, genuinely ambiguous case — a tool whose result could reasonably be shown one of two ways depending on how the question was phrased (a single-entity profile that might warrant just a stat card, or a stat card plus a trend chart, depending on whether the question asked about a

trend). Keep that judgment narrow and structurally separate from the deterministic lookup, so the overwhelming majority of tool results never pay for an LLM call just to decide how they should be displayed.

This is exactly the payoff of MCP tools returning structured data and never a UI directive: because the tool's output is plain structured data, this mapping layer can change — or the same result can be re-mapped to a different component entirely — without ever touching the tool itself.

7. Structural guardrails: required statements must not depend on the model remembering them

§2 and §3 are guardrails about *what the agent does*. A second class covers *what the answer must say*. Some domains carry statements that must accompany a result every single time: "this data source is a preliminary release," "this relationship is correlation, not proof of cause," "this projection is illustrative, not a forecast," "this is a licensed dataset, cite it as follows." A system prompt that asks the model to include them will work most of the time — and "most of the time" is exactly the failure rate a required disclosure cannot have.

Put required statements in **deterministic code that runs after the tools do**, not in the prompt:

```
# One fixed table: which statement each tool's result obliges the answer to carry
REQUIRED_NOTES = {
    "get_trend_vs_outcome":  ["Long-run co-movement, not proof of cause; not a climate model."],
    "get_scenario_outcomes": ["Illustrative translation of scenarios, not a projection."],
}

def required_notes(tool_calls):
    notes = []
    for call in tool_calls:
        if call.failed:
            continue                        # a failed call obliges nothing
        for note in REQUIRED_NOTES.get(call.tool_name, []):
            if note not in notes:           # de-duplicate across calls
                notes.append(note)
    return notes
```

The notes are attached to the response through a dedicated channel (rendered as an alert above the charts, say), so they appear regardless of what the model wrote. Three design details matter:

- **Fixed text, not a pass-through of the tool result's own caveat list.** Source-side caveat lists tend to run long and mix user-relevant statements with pipeline-internal and licensing prose. Keep the full list in what the *model* sees (so it can reason with it) and a short, curated, user-facing statement per tool in what the *user* sees.
- **Known gap to document, not hide:** a follow-up turn answered from earlier context with no new tool call attaches no notes, because no tool ran. Decide deliberately whether that is acceptable.

- **The prompt still carries the framing rules** ("describe correlation as co-movement, never attribute a global outcome to one actor"). Prompts shape tone; structure guarantees presence. Use both, and never rely on the prompt for the part that must not fail.

Routing examples are part of the guardrail. The classifier in §2 decides whether a question reaches the tool loop at all. If its prompt only shows examples of obviously data-shaped questions, a *relationship* question ("how is A related to B?") can be classified as general knowledge and answered from the model's memory, with no data and no required notes. Add examples of exactly these borderline questions to the classifier's prompt, on the data side — and distinguish them from conceptual ones ("what is X?"), which really are general knowledge.

8. Bounding what the model sees: the model view vs. the widget view

A result can be right for the chart and wrong for the model. A 150-year monthly series is what a line chart needs; it is thousands of tokens of context the model cannot use and will not read. Keep **two views of every tool result**:

View	Contents	Consumer
Full result	Every point/row, stored on the tool-call record (and in the thread's tool cache)	The widget that renders the chart
Model view	A capped copy: long series down-sampled to a fixed number of evenly spaced points (first and last always kept), bulky diagnostic blocks dropped, plus a note saying "you are seeing a sample"	The `ToolMessage` handed back to the model

```python
def cap_for_model(result: dict, max_points: int = 25) -> dict:
    capped = copy.deepcopy(result)              # never mutate the stored full result
    pts = capped.get("points", [])
    if len(pts) > max_points:
        idx = sorted({round(i * (len(pts) - 1) / (max_points - 1)) for i in range(max_points)})
        capped["points"] = [pts[i] for i in idx]
        capped["points_total"], capped["points_shown"] = len(pts), len(idx)
        capped["model_view_note"] = "Sampled for brevity; the user sees the full series."
    return capped
```

Apply the cap on **both** the fresh-call path and the cache-hit path (otherwise a repeated call re-inflates the context). Never cap the pre-computed summary or the caveat/attribution envelope — those are the parts the model is supposed to quote. And do the capping in the *agent*, not in the tool server: the server should stay consumer-agnostic, since a different client (a desktop app, say) may want the full series.

9. Compute the numbers; let the model phrase them

The most damaging class of agent bug in a data product is a **correct tool result turned into a wrong sentence**: a unit converted by the model ("2,751,504 $GtCO_2$" when the raw number was in megatonnes and the right figure was about 2,751 Gt), a rank restated from memory, a percentage recomputed with the wrong base. The cure is not a stronger instruction; it is to stop asking the model to do arithmetic on the way to the user:

1. **Pre-computed summaries in the tool result.** Each tool returns a small `summary` object (first and last value, change, rank, fitted slope with its interval, the unit it is in) calculated in Python from the data. The model quotes `summary`; it never derives a figure from a raw series. Make units explicit inside the summary (a `unit` field, and where raw points are in a smaller unit, a pre-converted block in the reporting unit).

2. **Answer blocks built from results, not written by the model.** Beyond the prose answer, a data answer usually wants a row of headline figures (KPI cards), a one-line source/scope statement under each chart, and a warning chip on illustrative content. Generate all three with deterministic code from the same tool results, using a fixed per-tool template:

Block	Source	Example
KPI cards (≤3)	`summary` fields; the frontend formats the number	"Per capita · 2024: 8.7 t"
Source line	A fixed template per tool, filled from the result	"Source: , 1990–2024 · territorial CO_2; land-use excluded"
Badge	A fixed string per tool	"Illustrative · implied outcomes, not projections"

3. **Deterministic leads where the sentence is fully determined.** If one tool result fully determines the headline sentence (a scenario translation, say), build that sentence — and its required companion note — in code from the result and skip the "compose a response" LLM call for that turn entirely. It is cheaper, faster, and cannot contradict the data. Everything else is still composed by the model, but the compose prompt changes from "don't restate numbers" to "state the key figures from each widget's `summary`; don't describe the chart."

4. **Rank among what was actually ranked.** "Largest of 215 countries" must use the count the rank was computed over (countries with data that year), not a headline constant such as the number of recognised countries. Return the denominator from the tool (`n_ranked`) instead of letting the agent assume it. Likewise, if a ranking is capped by one field, only build "top three" cards when the cap and the ranking field agree — otherwise the cards are a guess.

10. Follow-ups are fixed lookups, too

Suggested next steps ("Open this in the Forecasts page," "Next: how does this relate to warming?") are navigation and prompts, not generated content. Use a **fixed lookup keyed on the tools that ran in the turn**:

- **Links** — `{label, route}` pairs, at most three, de-duplicated by route, drawn only from routes that exist (a test should read the frontend's real route table and its anchors, so a rename on either side fails a test). Build them from *successful* calls only. When the destination page reads URL state (selected entities, a metric, a year), carry exactly the state that reproduces what the answer showed — and nothing the page cannot represent. If a page's picker only knows a subset of entities, mark each series in the tool result with whether it is in that subset and let the link carry only the flagged ones; otherwise the link opens a different view than the one the user just saw. Omit parameters equal to the page's default, and do not pin a year the user never named (the link would go stale).

- **Prompt chips** — up to three follow-up questions chosen per tool, skipping any equal to the current question and any whose tool already ran this turn. These are *prompts* (the user clicks to send or edit), a separate field from links.

Both are additive fields on the response: an older frontend ignores them, which keeps rollout order flexible.

11. Summary

Concern	Mechanism
Off-topic/subjective requests	Classify first, in a dedicated routing node, before the tool loop
Runaway tool-calling	A hard per-turn cap, checked on the routing edge, cache hits included
Repeated identical calls (same thread)	App-level cache, keyed on normalized arguments
Repeated identical request prefix (same provider)	Provider-side prompt caching on large, unchanging request components
Long-running loops	Stream templated, per-tool progress events, not a single blocking call
Rendering	Fixed tool → UI-intent lookup; LLM judgment reserved for genuinely ambiguous cases only
Statements an answer must carry	Deterministic notes attached after the tools run, from a fixed per-tool table
Large results	Two views: full result for the widget, capped sample for the model
Numbers in sentences	Computed in code (`summary`, KPI cards, source lines); the model only phrases them
Next steps	Fixed lookups for links (with URL state) and prompt chips, validated against the real frontend

See also

- Conversational Agent curriculum index
- Core Concepts, §5 for the loop this document bounds and instruments
- MCP Server, Tool Design for the guarantees this document assumes the tool source already provides
- Testing — verifying guardrail and cap behavior directly, not only end-to-end